

# Отчет об исследовании защитных механизмов: Реализация селективного спуфинга протокола Warden в среде World of Warcraft 3.3.5a (Build 12340)

## 1. Введение и архитектурный контекст

В сфере исследований безопасности игровых протоколов и защитных механизмов, система Warden, разработанная компанией Blizzard Entertainment, представляет собой один из наиболее долгоживущих и архитектурно сложных примеров динамической защиты клиента. Данный отчет посвящен детальному техническому анализу и реализации механизмов обхода (спуфинга) специфических проверок целостности — DRIVER\_CHECK, MPQ\_CHECK и PROC\_CHECK — в контексте клиента World of Warcraft версии 3.3.5a (Build 12340). Исследование проводится с позиции defensive security research, с целью изучения уязвимостей в логике валидации данных на стороне клиента и сервера, особенно в условиях модифицированных серверных ядер, используемых на проектах типа Warmane.<sup>1</sup>

### 1.1. Эволюция и назначение модуля Warden

Модуль Warden не является статической частью исполняемого файла WoW.exe. Вместо этого он представляет собой динамически загружаемую библиотеку (в формате PE-файла, но загружаемую вручную в память без регистрации в LDR-списках Windows), которая передается сервером клиенту во время сессии.<sup>3</sup> Эта архитектура позволяет разработчикам (или администраторам частных серверов) обновлять сигнатуры угроз, алгоритмы сканирования и смещения памяти в реальном времени, не требуя обновления самого игрового клиента. В версии 3.3.5a используется вторая или ранняя третья итерация протокола, характеризующаяся использованием потокового шифрования RC4 для защиты канала связи между модулем и сервером, а также использованием алгоритма SHA-1 для хеширования проверяемых сегментов памяти и файлов.<sup>3</sup>

Основная цель Warden — обеспечение целостности игровой среды путем обнаружения стороннего программного обеспечения, модификаций памяти (инъекций, хуков) и драйверов уровня ядра, которые могут использоваться для автоматизации игрового процесса или получения нечестного преимущества. Однако, архитектура, полагающаяся на доверие к клиентской стороне для выполнения проверок и отправки результатов, содержит фундаментальный недостаток: если клиентская среда скомпрометирована (например, через инъекцию DLL, как указано в постановке задачи), результаты проверок могут быть перехвачены и фальсифицированы до момента их шифрования и отправки на

сервер.

## 1.2. Техническая постановка задачи

В рамках данного исследования мы оперируем в контексте уже внедренной динамической библиотеки (x86 DLL) в адресное пространство процесса WoW.exe. Существующая инфраструктура уже обеспечивает успешный перехват входящих пакетов `SMSG_WARDEN_DATA` (Opcode `0x02E6`) и исходящих `CMSG_WARDEN_DATA` (Opcode `0x02E7`) через установку хука на функцию шифрования RC4.<sup>6</sup> Текущее состояние системы характеризуется режимом "pass-through": запросы парсятся для анализа, но ответы отправляются в оригинальном виде, что приводит к обнаружению любых модификаций клиента.

Задача состоит в переходе от пассивного мониторинга к активному вмешательству (MITM — Man-In-The-Middle) внутри процесса. Необходимо реализовать логику селективной подмены данных для трех критических типов проверок, которые часто используются для детекции продвинутых инструментов (таких как PQR, WPE Pro, EWT и драйверных обходов):

1. **DRIVER\_CHECK (Opcode 0x71):** Проверка наличия драйверов устройств по их именам в пространстве имен объектов Windows.<sup>7</sup>
2. **MPQ\_CHECK (Opcode 0x98):** Верификация целостности архивов MPQ (MoPaQ) для предотвращения модификаций моделей или текстур.<sup>8</sup>
3. **PROC\_CHECK (Opcode 0x7E):** Проверка целостности кода внешних или внутренних процедур (функций) для обнаружения сплайсинга (инлайн-хуков).<sup>9</sup>

Сложность реализации обусловлена не только необходимостью корректного формирования бинарной структуры ответа, но и особенностями реализации парсера строк в версии 3.3.5a, который содержит известные ошибки индексации, требующие точного воспроизведения ("bug-compatibility") для успешного прохождения валидации на стороне сервера.<sup>7</sup>

---

## 2. Архитектура протокола и криптографический слой

Понимание низкоуровневой структуры обмена данными является базисом для реализации любого механизма спуфинга. Протокол Warden работает поверх стандартного TCP-соединения World of Warcraft, инкапсулируя свои пейлоады в специальные опкоды.

### 2.1. Инкапсуляция и шифрование RC4

Все коммуникации Warden защищены симметричным потоковым шифром RC4. Ключ

сессии генерируется при начальном рукопожатии (handshake) на основе случайного семени (seed), передаваемого сервером, и сессионного ключа аккаунта (K), используя алгоритм SHA-1 для генерации финального 16-байтового ключа RC4.<sup>3</sup>

Процесс отправки ответа клиентом выглядит следующим образом:

1. Клиент формирует буфер с результатами проверок.
2. Вызывается функция RC4\_Encrypt (или её аналог внутри модуля), которая производит операцию XOR над данными буфера и псевдослучайной гаммой, генерируемой S-блоком RC4.
3. Зашифрованные данные упаковываются в пакет CMSG\_WARDEN\_DATA и отправляются на сервер.

Для реализации спуфинга критически важно производить модификацию данных **до** вызова функции шифрования. Если модифицировать данные после шифрования, сервер не сможет их расшифровать, так как нарушение целостности потока RC4 приведет к рассинхронизации состояния S-блока на сервере и клиенте, что неминуемо вызовет дисконнект или бан. Следовательно, хук должен быть установлен непосредственно на входе функции шифрования или в процедуре, формирующей пакет перед шифрованием.

## 2.2. Структура запроса WARDEN\_SMSG\_CHEAT\_CHECKS\_REQUEST

Сервер инициирует проверки, отправляя пакет с внутренним опкодом WARDEN\_SMSG\_CHEAT\_CHECKS\_REQUEST (обычно 0x02). Структура этого пакета является ключевой для понимания того, как интерпретировать входящие данные.

| Смещение | Тип данных | Описание                                             |
|----------|------------|------------------------------------------------------|
| 0x00     | uint8      | Опкод Warden (0x02 - Request Checks)                 |
| 0x01     | uint16     | Размер строкового пула (String Store Length)         |
| 0x03     | byte       | Строковый пул (набор null-terminated строк)          |
| 0x03+N   | byte       | Тело проверок (последовательность структур проверок) |
| End      | uint8      | XOR-ключ для                                         |

|  |  |                            |
|--|--|----------------------------|
|  |  | расшифровки типов проверок |
|--|--|----------------------------|

Строковый пул содержит имена модулей, драйверов, MPQ-архивов и процедур, которые будут проверяться. Одной из критических особенностей реализации версии 3.3.5a является то, как именно проверки ссылаются на эти строки.

## 2.3. Цикл обработки проверок и уязвимость индексации

После строкового пула следует массив проверок. Каждая проверка начинается с 1-байтового типа (Opcode), который обфусцирован с помощью однобайтового XOR-ключа, находящегося в самом конце пакета.<sup>3</sup>

Алгоритм парсинга выглядит так:

1. Прочитать байт типа проверки.
2. Применить XOR с ключом для получения реального типа (например, 0x71 для DRIVER\_CHECK).
3. В зависимости от типа, прочитать соответствующее количество байт данных (аргументов проверки).
4. Выполнить проверку.
5. Повторять до конца пакета.

### Критическая уязвимость реализации (The Index Desynchronization Bug):

Исследования, проведенные сообществом TrinityCore и AzerothCore, выявили фундаментальную ошибку в том, как модуль Warden версии 3.3.5a обрабатывает индексы строк. Проверки типа DRIVER\_CHECK, MPQ\_CHECK и LUA\_EVAL\_CHECK явно читают 1 байт из потока данных, который служит индексом в строковом пуле (0-based index). Однако проверка MODULE\_CHECK (Opcode 0xD9) **не читает** индекс строки из потока данных проверки, хотя имя модуля для неё обязательно присутствует в строковом пуле.<sup>7</sup>

Это создает ситуацию рассинхронизации:

- **Сервер:** Добавляет строку "WpePro.dll" (индекс 0) для MODULE\_CHECK и строку "Afd32uu" (индекс 1) для DRIVER\_CHECK.
- **Клиент:** При обработке MODULE\_CHECK не инкрементирует внутренний счетчик использованных строк (или логика итерации построена иначе), в результате чего последующий DRIVER\_CHECK, запрашивающий "индекс 0" (по мнению клиента), может получить строку, предназначенную для предыдущей проверки, или наоборот.

При реализации спуфера необходимо **эмулировать эту ошибку**. Если ваш парсер будет "умнее" и "правильнее", чем оригинальный клиент, вы будете подменять ответы не для тех проверок, что приведет к отправке хеша MPQ-файла в ответ на запрос проверки драйвера, что гарантированно приведет к бану. Ваша реализация должна точно

воспроизводить логику итерации по строкам, используемую в WardenWin::RequestChecks.<sup>7</sup>

## 3. Детальный анализ реализации DRIVER\_CHECK (0x71)

Проверка драйверов (DRIVER\_CHECK) направлена на обнаружение присутствия в системе загруженных драйверов уровня ядра, которые часто используются для скрытия процессов (rootkits) или прямого доступа к физической памяти (DMA) в обход API операционной системы.

### 3.1. Структура запроса и механизм проверки

После XOR-декодирования типа 0x71, структура данных проверки в потоке выглядит следующим образом:

| Поле         | Размер  | Описание                                             |
|--------------|---------|------------------------------------------------------|
| Seed         | 4 байта | Случайное число для инициализации хеширования (salt) |
| SHA1         | 20 байт | Ожидаемый хеш драйвера (в случае его обнаружения)    |
| String Index | 1 байт  | Индекс имени драйвера в строковом пуле               |

#### Механизм работы клиента:

Клиент извлекает имя драйвера по индексу (например, \\.\Afd32uu или просто Afd32uu) и использует Windows API функцию QueryDosDeviceA. Эта функция пытается получить информацию о символической ссылке MS-DOS.

- Если функция возвращает ненулевое значение, драйвер считается **обнаруженным**.
- Если функция возвращает 0 и GetLastError возвращает ERROR\_FILE\_NOT\_FOUND, драйвер считается **отсутствующим**.<sup>3</sup>

### 3.2. Стратегия спуфинга и формат ответа

Сервер ожидает подтверждения того, что проверка была выполнена. В случае отсутствия драйвера (легитимное состояние), клиент должен записать в выходной буфер специфический байт.

Анализ исходного кода эмуляторов (AzerothCore/TrinityCore), которые реверс-инжинирили поведение официального клиента, показывает, что "чистый" результат обозначается байтом **0xE9**.<sup>3</sup>

C++

```
// Фрагмент логики сервера (AzerothCore)
case DRIVER_CHECK:
{
    const uint8 byte = 0xE9;
    if (memcmp(buff.contents() + buff.rpos(), &byte, sizeof(uint8))!= 0) {
        // Логика обработки обнаружения чита (FAIL)
    }
    // Логика успешного прохождения (PASS)
}
```

#### Реализация подмены:

1. Перехватить обработку типа 0x71.
2. Прочитать и проигнорировать 4 байта Seed и 20 байт SHA1.
3. Прочитать 1 байт String Index.
4. **Важно:** Не выполнять реальный вызов QueryDosDeviceA. Это избыточно и может быть небезопасно, если анти-чит использует ловушки (honeypots).
5. Вместо результата проверки записать в выходной буфер один байт: 0xE9.

Использование байта 0xE9 может показаться странным (это опкод инструкции JMP в x86), но в контексте протокола Warden это магическая константа, сигнализирующая об отсутствии искомого объекта (драйвера, страницы памяти и т.д.). Любое другое значение будет интерпретировано сервером как аномалия или признак наличия драйвера, что повлечет за собой дальнейшие проверки или немедленную блокировку.

---

## 4. Детальный анализ реализации MPQ\_CHECK (0x98)

Проверка MPQ\_CHECK используется для контроля целостности игровых архивов. Это основной метод борьбы с изменением моделей (например, удаление стен, изменение размеров персонажей) и текстур, которые могут дать визуальное преимущество.

## 4.1. Структура запроса и алгоритм хеширования

Тип проверки 0x98 имеет следующую структуру данных:

| Поле         | Размер | Описание                                |
|--------------|--------|-----------------------------------------|
| String Index | 1 байт | Индекс имени файла MPQ в строковом пуле |

В отличие от DRIVER\_CHECK, здесь сервер не присылает ожидаемый хеш или Seed. Сервер просто указывает: "Проверь файл common.mpq".

### Механизм работы клиента:

Клиент получает имя файла (например, Data\common.mpq), открывает его и производит вычисление контрольной суммы. В версии 3.3.5a обычно используется алгоритм SHA-1 для всего файла или его критических заголовков. Однако, учитывая размер файлов (некоторые достигают гигабайтов), Warden часто проверяет не весь файл целиком за один раз, а использует технику выборочного хеширования или проверяет только заголовок архива и таблицу файлов. Но наиболее надежная реализация подразумевает полный хеш, который может кешироваться.

## 4.2. Стратегия спуфинга: Динамическая подмена

Сервер ожидает от клиента статус проверки и хеш файла. Согласно анализу серверной части <sup>3</sup>, структура ответа ожидается следующая:

1. **Результат (1 байт):** 0x00 (успех/файл найден).
2. **Хеш (20 байт):** SHA-1 хеш файла.

Если отправить только 0x00, сервер, скорее всего, отбросит пакет как некорректный, так как он ожидает данные для сверки с эталонной базой хешей.

### Проблема модифицированных клиентов:

На серверах типа Warmane часто используются кастомные патчи (например, patch-w.mpq для HD-моделей). Если ваша DLL работает в клиенте с этими патчами, "честный" хеш файла будет отличаться от стандартного хеша Blizzard 3.3.5a.

- Если сервер Warmane ожидает кастомный хеш, то простого проброса (pass-through) достаточно, и спуфинг не нужен, если вы не модифицировали этот конкретный файл.

- Если же вы используете удаление текстур (Wallhack через MPQ), то ваш локальный файл изменен.

#### Алгоритм реализации:

1. **Подготовка (Offline):** Необходимо заранее вычислить "чистые" SHA-1 хеши для всех стандартных MPQ-файлов версии 3.3.5a (Build 12340). Эти хеши должны быть жестко зашиты в вашу DLL или загружаться из внешнего конфигурационного файла.
2. **Перехват:** При получении запроса 0x98 извлечь имя файла.
3. **Поиск:** Найти соответствующий эталонный хеш в вашей базе данных.
4. **Формирование ответа:**
  - Записать байт 0x00.
  - Записать 20 байт эталонного хеша.

**Важное замечание по Warmane:** Поскольку Warmane распространяет свой собственный клиент с модифицированными MPQ (например, с новыми моделями из WoD/Legion), "чистым" хешем для них будет являться хеш их версии файла, а не оригинального Blizzard. Вам необходимо скачать чистый клиент Warmane, сгенерировать хеши всех MPQ и использовать их как эталон. Использование хешей от стандартного клиента 3.3.5a на сервере с кастомными патчами немедленно выдаст вас.

---

## 5. Детальный анализ реализации PROC\_CHECK (0x7E)

Проверка процедур (PROC\_CHECK) — это наиболее сложный и опасный тип проверки, направленный на обнаружение модификаций исполняемого кода в памяти (Code Injection, API Hooking, VTable hooks).

### 5.1. Структура запроса и целевые объекты

Тип проверки 0x7E содержит детальные инструкции по поиску функции в памяти:

| Поле | Размер  | Описание                   |
|------|---------|----------------------------|
| Seed | 4 байта | Инициализатор хеширования  |
| SHA1 | 20 байт | Ожидаемый хеш (для сверки) |

|              |         |                                                    |
|--------------|---------|----------------------------------------------------|
| Module Index | 1 байт  | Индекс имени модуля (например, WoW.exe, lua51.dll) |
| Proc Index   | 1 байт  | Индекс имени процедуры (функции)                   |
| Offset       | 4 байта | Смещение относительно начала функции               |
| Length       | 1 байт  | Количество байт для чтения и проверки              |

#### Механизм работы клиента:

1. Клиент получает дескриптор модуля через GetModuleHandleA.
2. Получает адрес функции через GetProcAddress.
3. Добавляет Offset к полученному адресу.
4. Читает Length байт из памяти процесса по этому адресу.
5. Отправляет прочитанные байты на сервер (или их хеш, в зависимости от подтипа, но чаще сырые байты для детального анализа).

## 5.2. Угроза хуков и стратегия "Shadow Copy"

Если вы используете инжектированную DLL, вы, вероятно, установили хуки (detours) на ключевые функции: Lua\_DoString, NetClient::Send, EndScene (для отрисовки). Стандартный метод установки хука (JMP-пачтинг) заменяет первые 5 байт функции на инструкцию JMP <адрес\_вашей\_функции>.

Когда Warden запрашивает PROC\_CHECK для Lua\_DoString с длиной 5 байт, он увидит ваш JMP вместо оригинального пролога функции (55 8B EC...). Это 100% бан.

#### Реализация подмены (Shadow Copy Strategy):

Для успешного обхода необходимо возвращать оригинальные байты функции, как если бы хука не существовало.

#### Методика:

1. **Инициализация:** При загрузке вашей DLL (в DllMain), до установки каких-либо хуков, создайте "теневую копию" (Shadow Copy) критических модулей или секций кода .text.
  - Можно скопировать только начала (первые 16-32 байта) всех экспортируемых функций lua51.dll и WoW.exe.

- Альтернативно, можно отобразить файл WoW.exe с диска в память повторно (используя `CreateFileMapping / MapViewOfFile`) в режиме `SEC_IMAGE`, но не исполнять его. Это даст вам доступ к "чистым" байтам кода, каким он лежит на диске.
- 2. **Перехват:** При обработке `0x7E`, разрешить адрес целевой функции.
- 3. **Переадресация чтения:** Вместо чтения из текущей (грязной) памяти процесса, вычислить соответствующий адрес в "теневой копии".
  - Формула:  $\text{ShadowAddress} = \text{ShadowBase} + (\text{TargetAddress} - \text{ImageBase})$ .
- 4. **Формирование ответа:**
  - Прочитать `Length` байт по `ShadowAddress`.
  - Записать `0x00` (статус успеха).
  - Записать прочитанные байты.

**Важное замечание по релокациям:** При использовании метода повторного маппинга файла с диска, необходимо учитывать, что если `WoW.exe` имеет таблицу релокаций (хотя основной модуль обычно грузится по фиксированному адресу `0x00400000`), то адреса внутри инструкций могут отличаться. Однако, для проверки прологов функций (которые обычно состоят из `push ebp; mov ebp, esp`) это редко является проблемой, так как они позиционно-независимы. Для `JMP/CALL` внутри кода релокации важны, но `PROC_CHECK` чаще всего проверяет именно целостность начала функции.

---

## 6. Синтез и интеграция в цикл обработки

Реализация спуфинга должна быть интегрирована в единый цикл обработки пакета `WARDEN_SMSG_CHEAT_CHECKS_REQUEST`. Ниже представлен псевдокод архитектуры обработчика, учитывающий все вышеописанные нюансы.

### 6.1. Псевдокод обработчика

C++

```
void InterceptAndSpoofWarden(ByteBuffer& packet) {  
    // 1. Читаем длину строк и сами строки  
    uint16 strLen = packet.readUInt16();  
    std::vector<std::string> strings = ParseStrings(packet, strLen);  
  
    // Подготовка выходного буфера  
    ByteBuffer resultBuff;
```

```

// 2. Цикл по проверкам
while (packet.readPos < packet.size() - 1) { // -1 для XOR ключа
    uint8 type = packet.readUInt8();
    uint8 xorKey = packet.peekLastUInt8(); // Ключ в конце пакета
    uint8 realType = type ^ xorKey;

    switch (realType) {
        case 0x71: // DRIVER_CHECK
        {
            // Читаем аргументы, чтобы сдвинуть указатель чтения
            packet.readUInt32(); // Seed
            packet.readBytes(20); // SHA1
            uint8 strIdx = packet.readUInt8(); // Index

            // SPOOF: Пишем 0xE9
            resultBuff.writeUInt8(0xE9);
            break;
        }
        case 0x98: // MPQ_CHECK
        {
            uint8 strIdx = packet.readUInt8();
            std::string filename = strings[strIdx];

            // SPOOF: Получаем чистый хеш из базы
            uint8* cleanHash = GetCleanMpqHash(filename);
            resultBuff.writeUInt8(0x00);
            resultBuff.append(cleanHash, 20);
            break;
        }
        case 0x7E: // PROC_CHECK
        {
            // Читаем аргументы
            packet.readUInt32(); // Seed
            packet.readBytes(20); // SHA1
            uint8 modIdx = packet.readUInt8();
            uint8 proclIdx = packet.readUInt8();
            uint32 offset = packet.readUInt32();
            uint8 len = packet.readUInt8();

            // SPOOF: Читаем из теневой копии
            std::vector<uint8> cleanBytes = ReadFromShadowCopy(strings[modIdx],
strings[proclIdx], offset, len);
            resultBuff.writeUInt8(0x00);

```

```

        resultBuff.append(cleanBytes);
        break;
    }
    case 0xD9: // MODULE_CHECK
    {
        // ВАЖНО: Эмуляция бага индексации.
        // MODULE_CHECK в 3.3.5a читает Seed(4) + SHA1(20), но HE читает индекс строки!
        // Однако строка для него есть в пуле.
        // Мы просто пропускаем данные, не трогая индексы других проверок.
        packet.readUInt32();
        packet.readBytes(20);

        // Spoofing результата: 0xE9 (как и driver check)
        resultBuff.writeUInt8(0xE9);
        break;
    }
    default:
    {
        // Для остальных проверок (TIMING, PAGE_CHECK, MEM_CHECK)
        // можно выполнить реальную проверку или тоже спуфить по необходимости.
        // Лучше выполнить оригинальную функцию проверки клиента для безопасности.
        RunOriginalCheck(realType, packet, resultBuff);
        break;
    }
}
}
}

// 3. Шифрование и отправка resultBuff
SendSpoofedResponse(resultBuff);
}

```

## 6.2. Управление памятью и производительность

При реализации MPQ\_CHECK следует избегать чтения гигабайтных файлов с диска в основном потоке игры (Main Thread), так как это вызовет заметный "фриз" (зависание) клиента, что может быть зафиксировано сервером через TIMING\_CHECK (проверка GetTickCount до и после выполнения батча проверок).<sup>11</sup>

### Рекомендация:

- Хеши MPQ должны быть предвычислены и загружены в память при старте.
- Теневая копия WoW.exe должна быть загружена при старте.
- Обработка пакета должна занимать микросекунды, не обращаясь к диску.

## 7. Контрмеры Warmane и эвристический анализ

Администрация Warmane известна тем, что использует кастомные конфигурации Warden и серверные эвристики. Спуфинг ответов Warden — это лишь первый слой защиты.

### 7.1. Проверки согласованности (Consistency Checks)

Сервер может запросить PROC\_CHECK для функции Lua\_DoString и одновременно PAGE\_CHECK (Opcode 0xB2) для страницы памяти, где эта функция находится. PAGE\_CHECK хеширует всю страницу (4KB). Если вы подмените ответ для PROC\_CHECK (вернув чистые байты пролога), но вернете реальный хеш "грязной" страницы для PAGE\_CHECK (или наоборот), сервер обнаружит несоответствие.

- **Вывод:** Необходимо спуфить **все** типы проверок, затрагивающих память (MEM\_CHECK, PAGE\_CHECK\_A/B, PROC\_CHECK), используя единый источник "чистых" данных (Shadow Copy).

### 7.2. Серверные проверки поведения

Даже идеальный спуфинг Warden не спасет от поведенческого анализа. Если PQR отправляет пакеты заклинаний с нечеловеческой реакцией (0ms delay), или если вы используете Unlocker, который посылает защищенные Lua-команды, сервер забанит вас по логам действий, игнорируя "чистые" ответы Warden. Спуфинг Warden скрывает инструмент, но не действия.

## 8. Заключение

Реализация селективного спуфинга Warden для WoW 3.3.5a — это сложная задача, требующая глубокого понимания не только криптографии и структуры пакетов, но и специфических багов реализации конкретной версии клиента (Build 12340). Ключевым фактором успеха является точность эмуляции: использование магического байта 0xE9 для DRIVER\_CHECK, корректная работа со строковыми индексами в присутствии MODULE\_CHECK, и использование стратегии "Shadow Copy" для возврата оригинального кода при PROC\_CHECK. Для среды Warmane критически важно использовать эталонные хеши именно от их модифицированных файлов, чтобы избежать детекции на уровне несоответствия версий ресурсов. Представленная архитектура позволяет полностью скрыть факт наличия инжектированной DLL и установленных перехватов функций от штатных механизмов сканирования Warden.

### Works cited

1. How are bots detected in private servers? : r/wowservers - Reddit, accessed February 17, 2026, [https://www.reddit.com/r/wowservers/comments/9mzypr/how\\_are\\_bots\\_detected\\_in\\_private\\_servers/](https://www.reddit.com/r/wowservers/comments/9mzypr/how_are_bots_detected_in_private_servers/)

2. PQR,EWT Scripts block · Issue #2004 · azerothcore/azerothcore-wotlk - GitHub, accessed February 17, 2026,  
<https://github.com/azerothcore/azerothcore-wotlk/issues/2004>
3. AzerothCore WardenWin Implementation | PDF | Gnu | Software Development - Scribd, accessed February 17, 2026,  
<https://www.scribd.com/document/582975372/WardenWin>
4. Warden - OldGeneral discussion - getMaNGOS | The home of MaNGOS, accessed February 17, 2026,  
<https://www.getmangos.eu/forums/topic/1395-warden/>
5. Warden.h File Reference - AzerothCore Doxygen, accessed February 17, 2026,  
[https://www.azerothcore.org/doxygen/d7/dc7/Warden\\_8h.html](https://www.azerothcore.org/doxygen/d7/dc7/Warden_8h.html)
6. WoWTools/src/WoWPacketViewer/Parsers/Warden ... - GitHub, accessed February 17, 2026,  
[https://github.com/tomrus88/WoWTools/blob/master/src/WoWPacketViewer/Parsers/Warden/MSG\\_WARDEN\\_DATA.cs](https://github.com/tomrus88/WoWTools/blob/master/src/WoWPacketViewer/Parsers/Warden/MSG_WARDEN_DATA.cs)
7. warden module interprets the check incorrectly · Issue #28138 - GitHub, accessed February 17, 2026,  
<https://github.com/TrinityCore/TrinityCore/issues/28138>
8. warden\_checks | TrinityCore MMo Project Wiki, accessed February 17, 2026,  
[https://trinitycore.info/database/335/world/warden\\_checks](https://trinitycore.info/database/335/world/warden_checks)
9. packet package - github.com/superp00t/gophercraft/packet - Go Packages, accessed February 17, 2026,  
<https://pkg.go.dev/github.com/superp00t/gophercraft/packet>
10. TrinityCore/src/server/game/Scripting/ScriptReloadMgr.cpp at master - GitHub, accessed February 17, 2026,  
<https://github.com/TrinityCore/TrinityCore/blob/master/src/server/game/Scripting/ScriptReloadMgr.cpp>
11. warden\_checks | AzerothCore, accessed February 17, 2026,  
[https://www.azerothcore.org/wiki/warden\\_checks](https://www.azerothcore.org/wiki/warden_checks)